API DESIGN: ВІД «ЗООПАРКУ» ДО КОНТРАКТУ

REST, problem+json, пагінація, версії, OpenAPI, idempotency, contract testing

Заняття №9

Node.js Pro

Лектор
Кирило Орлов

r_d

ДРАБИНКА: 8 КРОКІВ · ОДИН ДОМЕН /orders · ПОРТИ 20901–20908

Перша половина — форма відповіді	Друга половина — еволюція і гарантії
1 «зоопарк» — API без правил	5 версії: v1 і v2 поруч
2 ресурси: іменники + статуси	6 OpenAPI з декораторів
3 problem+json — одна форма помилок	7 Idempotency-Key на POST
4 пагінація: offset проти cursor	8 контракт перевіряє машина, не рев'ю

НАСКРІЗНА НИТКА
API — це UI для чужого розробника, і він тобі не подзвонить. Кожен крок драбинки переносить ще одну домовленість із голови в машиночитний контракт — на кроці 8 його перевіряє скрипт з exit-кодом для CI.

ПЛАН ЗАНЯТТЯ



- Глосарій на старті + драбинка з восьми кроків
- Зоопарк → ресурси → problem+json — демо A
- Пагінація offset/cursor + Idempotency-Key — демо B
- Версії поруч: Deprecation · Sunset · Link
- OpenAPI з декораторів: спека народжується з коду
- Contract-check проти живого сервера — демо C + ДЗ

ГЛОСАРІЙ НА СТАРТІ: ШІСТЬ СЛІВ ЛЕКЦІЇ

01. resource

іменник в URL: /orders. Дії виражають методи, не назви

02. problem+json

єдина форма помилки за RFC 9457, з власним content-type

03. cursor

закладка «після елемента (created_at, id)», а не зсув на N

04. Idempotency-Key

ключ СПРОБИ у заголовку: повтор POST не списує вдруге

05. OpenAPI

машиночитний опис API; у нас — генерується з декораторів

06. contract-check

скрипт: живі відповіді проти спеки, exit-код для CI

КРОК 1 · «ЗООПАРК»: ПРАЦЮЄ, АЛЕ КОРИСТУВАТИСЬ НЕМОЖЛИВО

- кожен ендпойнт наче писав інший розробник в інший день
- дієслова в URL: /getAllOrders, /order_create — назви вгадуй
- видалення через GET: safe-метод, який мутує стан
- помилка їде зі статусом 200 — моніторинг по статусах сліпий

Це не карикатура, а реальне легасі: технічно все «працює», тести зелені. Незручно лише всім, хто цим API користується.

```
 GET /orders/delete?id=2
 → {"success":true}
[200]

 GET /orders/delete?id=2
 → {"success":false,
    "msg":"order not found"}
[200]

 POST /order_create  {}
 → {"success":false,
    "error_message":
      "EMPTY_ITEMS"}
[200]

_____________________________

помилка = 200 → моніторинг
по статусах СЛІПИЙ

GET, який видаляє, — міна:
краулер і префетч «клікають»
```

КРОК 2 · РЕСУРСИ: ІМЕННИКИ, МЕТОДИ, СТАТУСИ

```
// 02-resources.ts — той самий домен за правилами REST
@Controller('orders')
class OrdersController {
  @Post()
  @HttpCode(201)                        // створено — це 201, не 200
  create(@Body() dto: CreateOrderDto,  // валідація — з L#4 і L#7
         @Res({ passthrough: true }) res: Response) {
    const order: Order =
      { id: nextId++, items: dto.items, status: 'new', total: 0 };
    orders.push(order);
    res.setHeader('Location', `/orders/${order.id}`);
    return order;                       // 201 + де живе ресурс
  }

  @Delete(':id')
  @HttpCode(204)                        // видалено — тіла немає
  remove(@Param('id', ParseIntPipe) id: number) {
    const i = orders.findIndex((o) => o.id === id);
    if (i === -1)
      throw new NotFoundException(`замовлення ${id} не знайдено`);
    if (orders[i].status === 'paid')   // бізнес-правило → 409
      throw new ConflictException({ error_code: 'ALREADY_PAID',
        hint: 'спочатку зроби refund' });  // ТРЕТЯ форма помилки
    orders.splice(i, 1);
  }
}
```

Той самий домен — тепер за правилами REST

1 · ресурс /orders: дії виражає HTTP-метод

2 · 201 + Location на створення, 204 на видалення

3 · валідація DTO — вже була на L#4 і L#7

curl -i -X POST /orders → 201 · Location: /orders/3

але помилки досі мають ТРИ різні форми — біль лишився

ОДИН СЕРВЕР — ТРИ РІЗНІ ФОРМИ ПОМИЛОК

ВІДПОВІДЬ КРОКУ 2

404 → { message, error, statusCode }
400 → message раптом МАСИВ рядків
409 → { error_code, hint } — самодіяльність
статуси правильні, а форма щоразу інша
content-type скрізь один: application/json

ЩО РОБИТЬ КЛІЄНТ

- перший парсер: читає message-рядок
- другий парсер: обходить масив
- третій парсер: шукає error_code
- три гілки if у кожному клієнті
- помилку від даних відрізняє лише статус

Лікування — крок 3: один глобальний exception filter надає всім помилкам одну форму problem+json (RFC 9457) із власним content-type. Клієнт пише ОДИН парсер і матчить помилки по полю type, а не по тексту.



КРОК 3 · PROBLEM+JSON: АНАТОМІЯ ОДНІЄЇ ПОМИЛКИ

- type — стабільний URI: клієнт матчить ПО НЬОМУ, не по тексту
- title — один для всього класу; detail — про конкретний випадок
- instance — який саме запит породив помилку
- розширення дозволені: errors[] у валідації, code у конфліктах
- RFC 9457 (замінив RFC 7807) — стандарт, не наша вигадка
- один exception filter — і жодних трюків у контролерах

```
$ curl -s -i :20903/orders/99

HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
 "type": ".../problems/404",    ← клас
 "title": "Ресурс не знайдено", ← класу
 "status": 404,
 "detail": "замовлення 99 не знайдено",
 "instance": "/orders/99"       ← запит
}
```



```
// 03-problem.ts — ОДИН @Catch() на всі помилки застосунку
@Catch()
class ProblemFilter implements ExceptionFilter {
  catch(exception: unknown, host: ArgumentsHost) {
    const status = exception instanceof HttpException
      ? exception.getStatus() : 500;
    // …рядок | об'єкт → detail + extra (code, errors[])…
    res.status(status)
       .type('application/problem+json')  // власний content-type
       .json({
         type: `${PROBLEM_BASE}/${extra.code ?? status}`,
         title: TITLES[status] ?? 'Помилка',
         status, detail,                   // клас + випадок
         instance: req.originalUrl,        // який запит
         ...extra });
  }
}
// bootstrap: app.useGlobalFilters(new ProblemFilter())
```



```
// крок 4: офсетний дрейф, відтворений руками (:20904)

$ curl ':20904/orders?limit=3&offset=0'
  items: [25, 24, 23]   total: 25         // сторінка 1
     …хтось створює замовлення — воно стає рядком 0…
$ curl ':20904/orders?limit=3&offset=3'
  items: [23, 22, 21]                     // 23 — ВДРУГЕ

// той самий сценарій, але через cursor:
$ curl ':20904/orders?limit=3'
  items: [26, 25, 24]   next_cursor: "eyJjIjoiMjAyNi0…"
     …вставка…
$ curl ':20904/orders?limit=3&cursor=eyJjIjoiMjAyNi0…'
  items: [23, 22, 21]                     // без дубля

$ curl ':20904/orders?cursor=zzz'   →   [400]
```



КРОК 4 · CURSOR: «ПІСЛЯ ЕЛЕМЕНТА», А НЕ ЗСУВ

offset — «зріз від числа»: вставка зверху зсуває все під ногами

cursor кодує (created_at, id) останнього елемента — base64url

битий cursor → 400: він непрозорий і належить серверу

next_cursor: null = сторінок більше немає — клієнт не рахує їх сам. У SQL та сама закладка стає WHERE (created_at, id) < ($1, $2) — на L#12 побачимо, що на глибоких сторінках це ще й швидше за OFFSET, а COUNT(*) для total не безплатний.



```
// 04-pagination.ts — курсор: усе, що ПІСЛЯ (createdAt, id)
function encodeCursor(o: Order): string {
  return Buffer.from(JSON.stringify({ c: o.createdAt, id: o.id }))
               .toString('base64url');    // непрозорий для клієнта
}
let slice = all;
if (cursor) {
  const { c, id } = decodeCursor(cursor); // битий → 400
  slice = all.filter((o) =>
    o.createdAt < c || (o.createdAt === c && o.id < id));
}
const items = slice.slice(0, limit);
const last = items[items.length - 1];
return {
  items,
  next_cursor: items.length === limit && last
    ? encodeCursor(last)                 // закладка ПІСЛЯ останнього
    : null,                              // null = сторінок немає
};
```

КРОК 5 · ВЕРСІЇ ПОРУЧ + МАШИНОЧИТНЕ ПРОЩАННЯ

Deprecation

@1787184000 — «застаріло з 2026-08-20» (RFC 9745, @unix-час)

Sunset

Thu, 19 Nov 2026 — «після цієї дати вимкнемо» (RFC 8594)

Link

</v2/orders>; rel="successor-version" — куди мігрувати

Чому дві версії, а не фікс на місці

У v1 total — рядок "2600.00": клієнти парсять його float-ом і втрачають копійки. Виправити «на місці» = зламати всіх одразу. Тому v1 заморожена і прощається заголовками, а v2 поруч віддає total_cents: 260000 (int) + currency + created_at. Без версії — 404: поверхні /orders не існує.

ВЕРСІЮ — В URI ЧИ В ЗАГОЛОВОК?

Аспект	URI: /v1/orders	Header: Accept-Version
Видимість	видно в логах, curl, кеші	треба зазирати в заголовки
Кешування	ключ кешу = URL, працює саме	потрібен Vary: Accept-Version
Копіпаст запиту	посилання самодостатнє	без -H — мовчки інша версія
Чистота URI	версія «забруднює» шлях	URI стабільний назавжди
У Nest	VersioningType.URI — наш вибір	VersioningType.HEADER — те саме

КРОК 6 · OPENAPI: ДОКУМЕНТАЦІЯ, ЯКА НЕ БРЕШЕ

01. джерело правди — код: рукописна спека мертва за тиждень

02. DTO, що валідує (L#7), через @ApiProperty стає схемою — вона ОБІЦЯЄ, а не примушує

03. @ApiOkResponse / @ApiNotFoundResponse — КОЖЕН статус описано

04. /docs — живий Swagger UI із Try it out

05. /docs-json — контракт: Postman, кодоген клієнта, крок 8

06-OPENAPI.TS: ОДНА АНОТАЦІЯ — ДВІ РОБОТИ

```
class CreateOrderDto {
  @ApiProperty({ example: ['клавіатура', 'килимок'],
                 type: [String] })        // → схема компонента
  @IsArray() @ArrayNotEmpty() @IsString({ each: true })
  items!: string[];                       // → і валідація (L#7)
}

const problem = (d: string) => ({ description: d, content: {  // помилка — теж
  'application/problem+json': {           // контракт, і медіа-тип у неї СВІЙ
    schema: { $ref: getSchemaPath(ProblemDto) } } } });

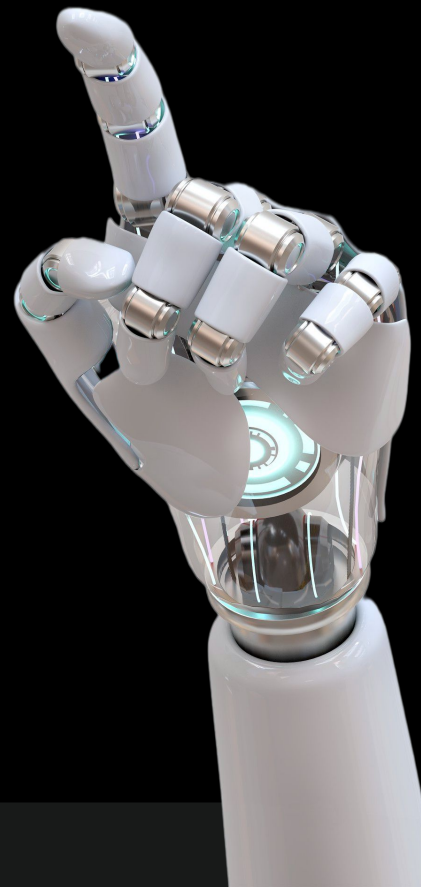
@ApiTags('orders') @ApiExtraModels(ProblemDto) @Controller('orders')
class OrdersController {
  @Get(':id')  @ApiOkResponse({ type: OrderDto })
  @ApiNotFoundResponse(problem('Замовлення з таким id немає'))
  one(@Param('id', ParseIntPipe) id: number): OrderDto { … }
}
// bootstrap: useGlobalFilters(new ProblemFilter())  ← фільтр кроку 3
const doc = SwaggerModule.createDocument(app, config);
SwaggerModule.setup('docs', app, doc, { jsonDocumentUrl: 'docs-json' });
$ curl -s :20906/docs-json → openapi 3.0.0 · /docs → 200 · 404 → problem+json
```



КРОК 7 · IDEMPOTENCY-KEY: П'ЯТЬ СЦЕНАРІЇВ ОДНОГО POST

Сценарій	Що робить сервер	Відповідь
без ключа	виконує як звичайний POST — без гарантій	201 щоразу
новий ключ	виконує; зберігає fingerprint тіла + відповідь	201
ключ + те саме тіло	НЕ виконує — віддає збережену відповідь	201 + Idempotency-Replay: true
ключ + ІНШЕ тіло	відхиляє: клієнт помилково реюзає ключ	422 problem+json
ключ «у польоті»	конкурентний повтор програє атомарному claim	409 — почекай

```
// 07-idempotency.ts — серце інтерсептора
const fingerprint = createHash('sha256')
  .update(JSON.stringify(req.body ?? null)).digest('hex');
const claimed = await this.store.claim(key,
  { state: 'in-flight', fingerprint });    // Redis: SET NX EX
if (!claimed) {                            // ключ уже існує
  const rec = await this.store.get(key);
  if (!rec) return next.handle();          // ключ щойно протух
  if (rec.fingerprint !== fingerprint)     // інше тіло!
    throw new UnprocessableEntityException(…);      // 422
  if (rec.state === 'in-flight')           // ще виконується
    throw new ConflictException(…);                 // 409
  res.setHeader('Idempotency-Replay', 'true');
  return of(rec.body);                     // збережена відповідь
}
return next.handle().pipe(mergeMap(async (body) => {
  await this.store.finish(key,
    { state: 'done', fingerprint, status: 201, body });
  return body;                             // перша спроба — виконали
}));
```



СХОВИЩЕ КЛЮЧІВ: ПАМ'ЯТЬ УМИРАЄ, REDIS — НІ

сховище за інтерфейсом: claim · finish · get — дві реалізації

MemoryStore — Map у процесі: рестарт = усі ключі втрачено

RedisStore — SET NX + EX: атомарний «хто перший» із TTL

TTL ~24 години — стільки тримає ключі Stripe

REDIS_URL → npm run step7 бере спільне сховище (порт 20990)

після kill + рестарту: replay працює, у лозі один «СТВОРЕНО»

«КНОПКА ЛІФТА»: IDEMPOTENCY ЯК АНАЛОГІЯ

ЧАСТИНА API

- перший POST із ключем — замовлення створено
- повтор із тим самим ключем — збережена відповідь
- Idempotency-Replay: true у заголовку
- нова спроба = НОВИЙ ключ від клієнта

АНАЛОГІЯ (ЛІФТ)

- натиснув «виклик» — ліфт поїхав
- тиснеш ще — другий ліфт НЕ приїде
- кнопка світиться: виклик уже прийнято
- нова поїздка = нове натискання

Де аналогія ламається: кнопка не знає, ЩО саме ти викликаєш, — а сервер звіряє fingerprint тіла: той самий ключ з іншим тілом → 422. І кнопка пам'ятає виклик лише до приїзду ліфта, а ключ живе TTL ~24 години.

КРОК 8 · КОНТРАКТ ПЕРЕВІРЯЄ МАШИНА, НЕ РЕВ'Ю

contract/check.mjs тягне /docs-json із ЖИВОГО сервера, робить 4 реальні запити («очікування консюмера») і валідує кожну відповідь Ajv-ом проти схем зі спеки — плюс звіряє content-type і задекларовані заголовки

DRIFT=1 — «невинний рефакторинг»: у v2 total_cents стає totalCents, спеку ніхто не чіпав. Контракт червоний: must have required property 'total_cents' — exit 1. У CI це зупинило б деплой ДО того, як роз'їзд упіймають клієнти

⚠ Це schema-validation як контракт: перевіряє ФОРМУ відповідей, швидко і без брокера. Consumer-driven контракти (Pact: очікування пише КЛІЄНТ, broker, can-i-deploy) — сьогодні лише поняття, practical wiring у CI — L#16

```
// contract/check.mjs (скорочено) — живий сервер проти ВЛАСНОЇ спеки
const spec = await (await fetch(`${BASE}/docs-json`)).json();
const CHECKS = [                    // «очікування консюмера»: 4 запити
  { req: { method: 'GET', url: '/v1/orders' },
    expect: { path: '/v1/orders', method: 'get', status: 200 } },
  // так само: GET /v2/orders 200 · POST /v1/orders 201 · /v1/orders/999999 404
];   let failed = 0;
for (const check of CHECKS) {
  const { path, method, status } = check.expect;
  const declared = spec.paths[path][method].responses[status];
  const res = await fetch(`${BASE}${check.req.url}`, check.req);
  // 1 статус · 2 content-type · 3 заголовки · 4 тіло (Ajv)
  const schema = declared.content[Object.keys(declared.content)[0]].schema;
  if (!ajv.compile(deref(schema, spec))(await res.json())) failed++;
}
process.exit(failed === 0 ? 0 : 1);   // exit-код — це CI-гейт

$ node contract/check.mjs             → 4/4 зелені, exit 0
$ DRIFT=1 npm run step8   +   npm run contract
  ✗ must have required property 'total_cents'  → exit 1
```

// що свідомо лишаємо за дужками — і де воно чекає далі

Pact у CI: broker, can-i-deploy, verification провайдера
    → сьогодні: поняття + різниця зі schema-validation
    → practical wiring у CI — L#16

GraphQL і gRPC як альтернативи REST
    → чому існують — сказали; deep dive — L#17 і L#21

Пагінація в SQL: WHERE (created_at, id) < ($1, $2)
    → сьогодні той самий ефект на масиві в пам'яті
    → EXPLAIN, індекси і числа — L#12

HATEOAS і рівень 3 за Річардсоном
    → зупиняємось на рівні 2: ресурси + методи + статуси

Auth (L#24), rate limiting (L#23), security hardening (L#27)
    → це теж API, але інший шар — окремі лекції

ПІДСУМКИ + МІСТ ДО L#10

ФОРМА

ресурси + статуси + problem+json: клієнт пише ОДИН парсер

ЕВОЛЮЦІЯ

cursor-пагінація та v1/v2 із Deprecation/Sunset: API росте, клієнти живі

ГАРАНТІЇ

повтор безпечний, спека виконувана — Idempotency-Key + contract-check

ДЗ №9

7 балів · дедлайн 27.08 · OpenAPI-спека курсового: Idempotency-Key + cursor-пагінація + problem+json + Pact-тест АБО runtime-валідація

ДАЛІ

L#10 — тестування: піраміда, Jest/Vitest. Contract-check — вже майже тест